实验1 VSCode C++ 开发环境构建

1.1 远程软件开发环境搭建

1.1.1 实验目的

了解如何使用 VSCode 远程连接 Linux 服务器（以 Ubuntu 为例），并在远程环境中搭建 C++ 开发环境。

1.1.2 实验内容与要求

要求

- 掌握 VSCode 远程连接 Linux 的方法。
- 在 VSCode 远程环境中搭建 C++ 开发环境。

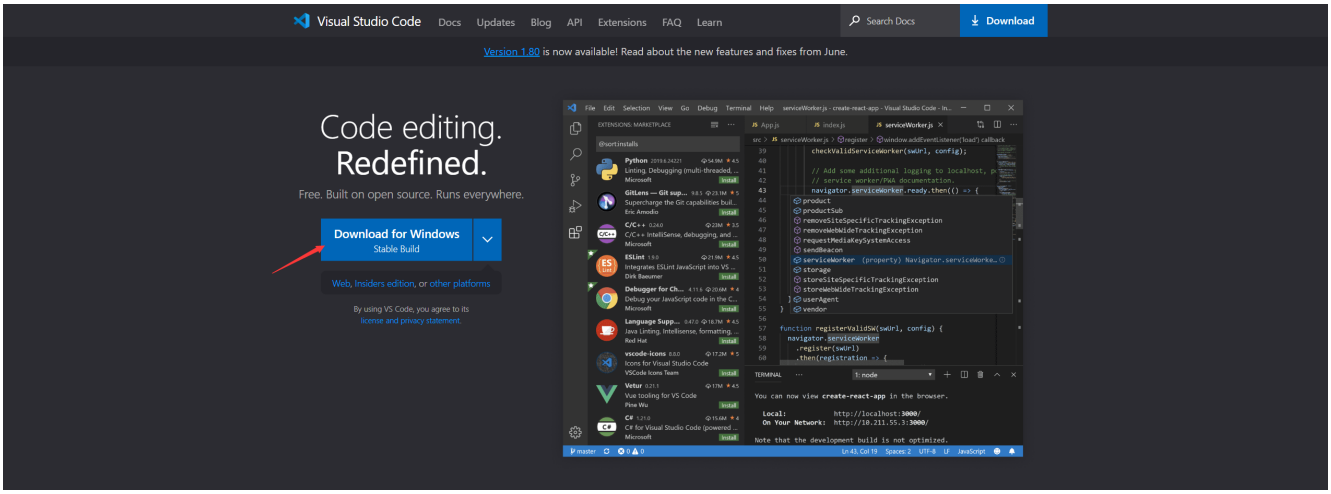
环境

1. Windows 桌面终端一台。
2. Ubuntu 服务器一台。

内容

一、在 Windows 上安装 VSCode

1. 前往 Visual Studio Code 官网：https://code.visualstudio.com/。
2. 下载适用于你操作系统的版本（Windows），双击下载的安装程序（.exe 文件）。
3. 然后按照安装向导进行安装。



二、配置 Ubuntu 服务器（远程登录和C++编译环境）

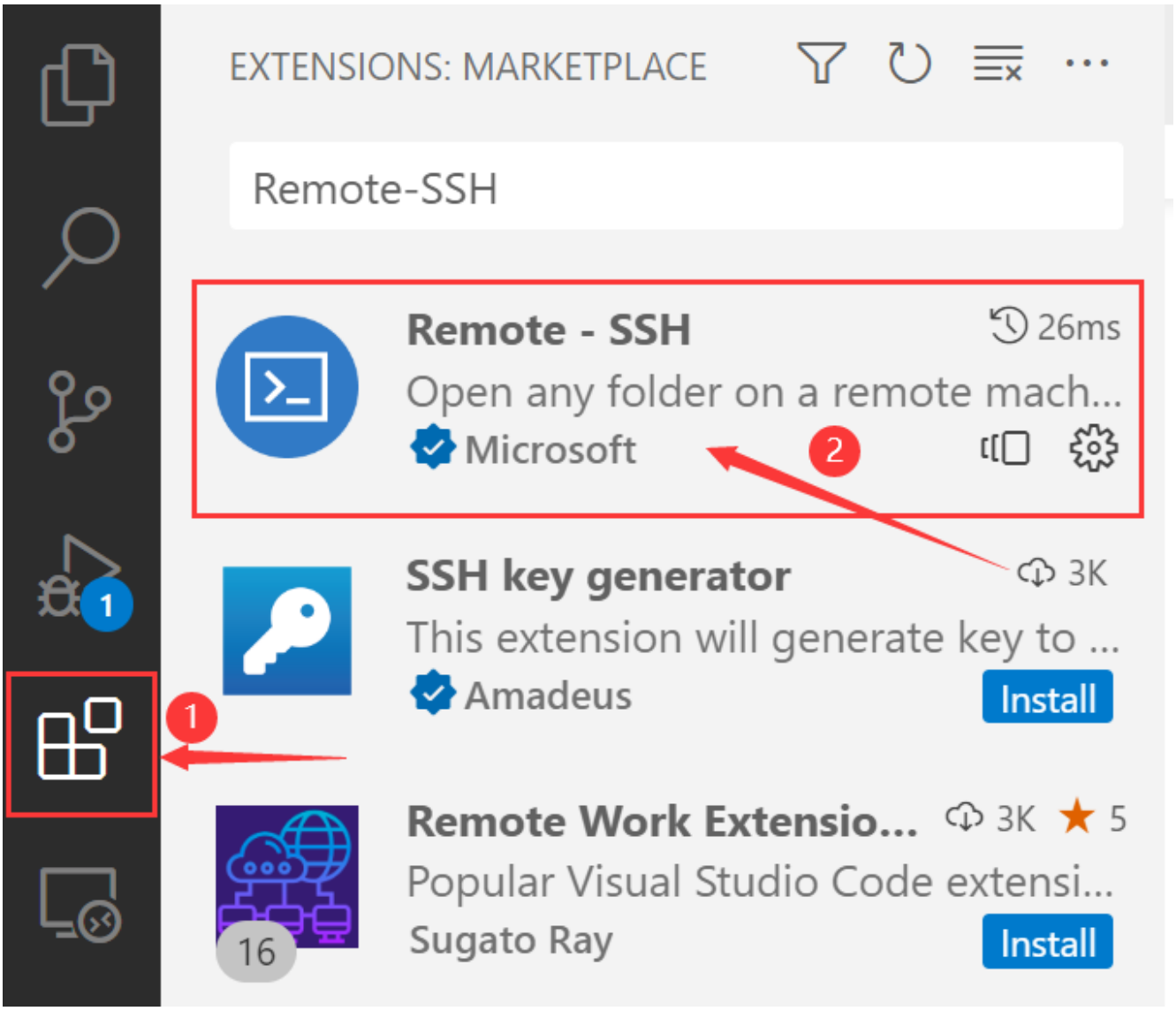
1. 更新软件源：`sudo apt update`。
2. 安装 ssh 服务：`sudo apt install openssh-server`。
3. 确保在 Windows 上可以通过 ssh 登录到 Ubuntu。
4. 安装 gcc 工具链：`sudo apt install build-essential gdb`。

5. 确认 gcc 安装成功：`gcc -v`。

```
usr@usr-virtual-machine:~$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/11/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none:amdgcn-amdhsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 11.3.0-1ubuntu1~22.04.1' --with-bugurl=file:///usr/share/doc/gcc-11/README.Bugs --enable-languages=c,ada,c++,go,brig,d,fortran,objc,obj-c++,m2
 --prefix=/usr --with-gcc-major-version-only --program-suffix=-11 --program-prefix=x86_64-linux-gnu- --enable-shared --enable-linker-build-id --libexecdir=/usr/lib --without-included-gettext --enable-threa
ds=posix --libdir=/usr/lib --enable-nls --enable-bootstrap --enable-clocale=gnu --enable-libstdcxx-debug --enable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enable-gnu-unique-object --disable-vt
able-verify --enable-plugin --enable-default-pie --with-system-zlib --enable-libphobos-checking=release --with-target-system-zlib=auto --enable-objc-gc=auto --enable-multiarch --disable-werror --enable-cet
 --with-arch-32=i686 --with-abi=m64 --with-multilib-list=m32,m64,mx32 --enable-multilib --with-tune=generic --enable-offload-targets=nvptx-none=/build/gcc-11-aYxV0E/gcc-11-11.3.0/debian/tmp-nvptx/usr,amdgc
n-amdhsa=/build/gcc-11-aYxV0E/gcc-11-11.3.0/debian/tmp-gcn/usr --without-cuda-driver --enable-checking=release --build=x86_64-linux-gnu --host=x86_64-linux-gnu --target=x86_64-linux-gnu --with-build-config
=bootstrap-lto-lean --enable-link-serialization=2
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 11.3.0 (Ubuntu 11.3.0-1ubuntu1~22.04.1)
usr@usr-virtual-machine:~$
```

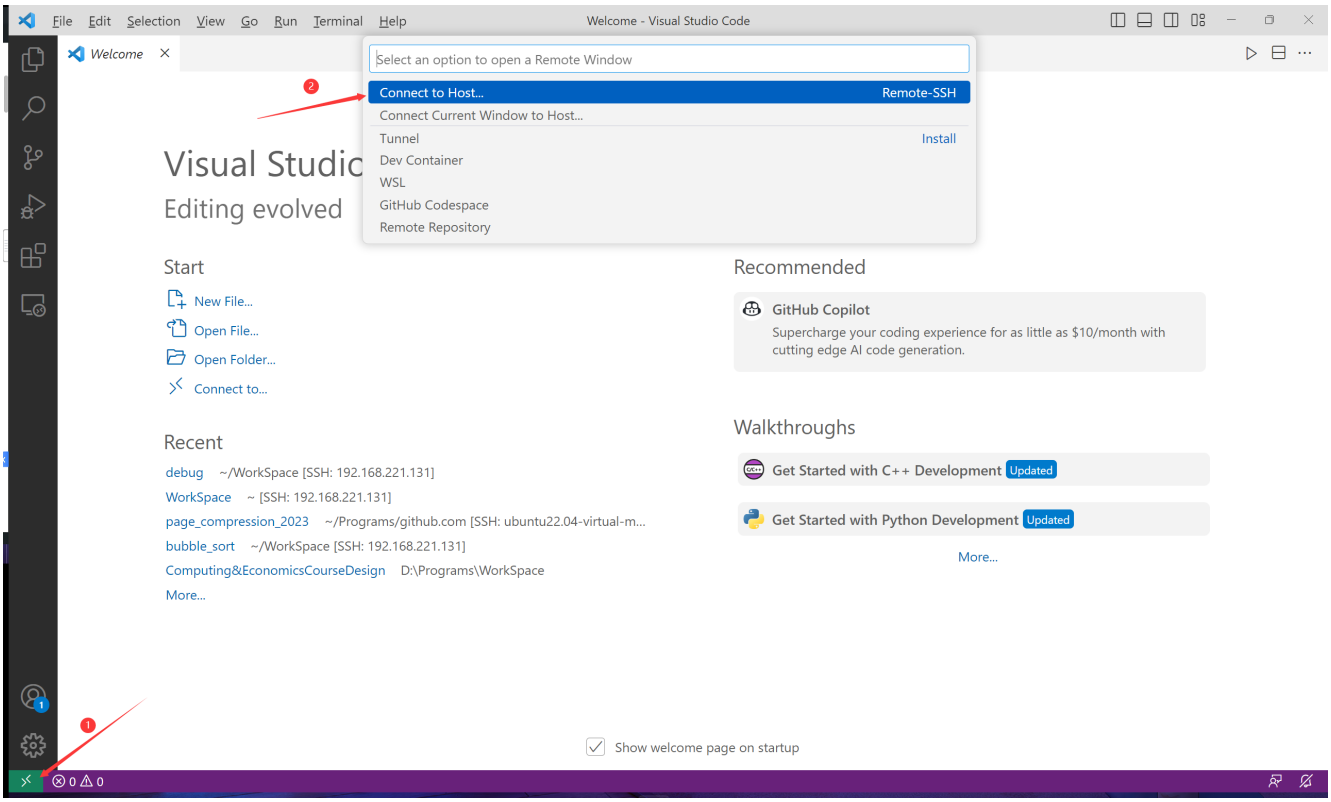
三、安装 VSCode 远程连接插件

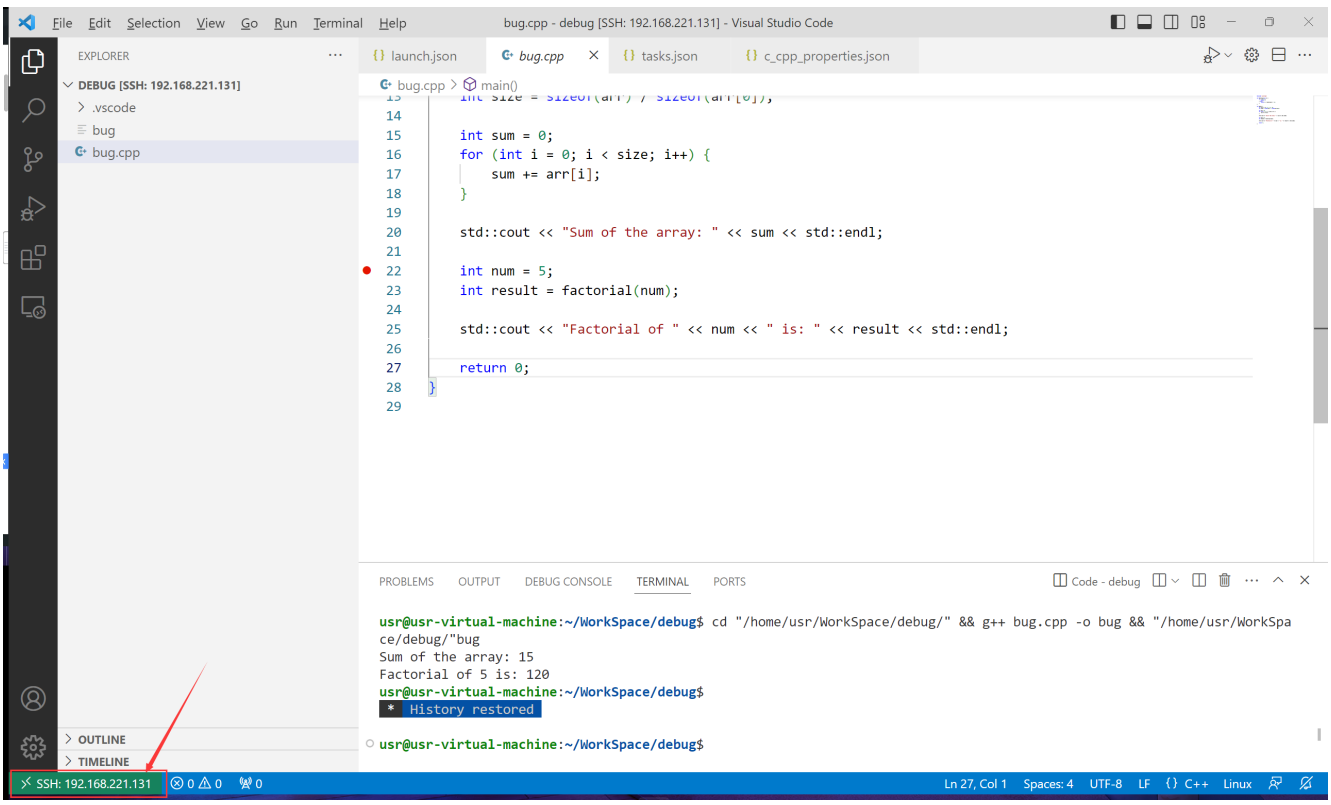
1. 打开VSCode。
2. 按下快捷键 `<Ctrl-Shift-x>` 或点击侧边栏的 "Extensions" 搜索插件。
3. 在搜索框中输入 "Remote-SSH" 并安装该插件。
4. 安装完成后，重新启动VSCode。



四、VSCode 通过 SSH 远程连接 Ubuntu 服务器

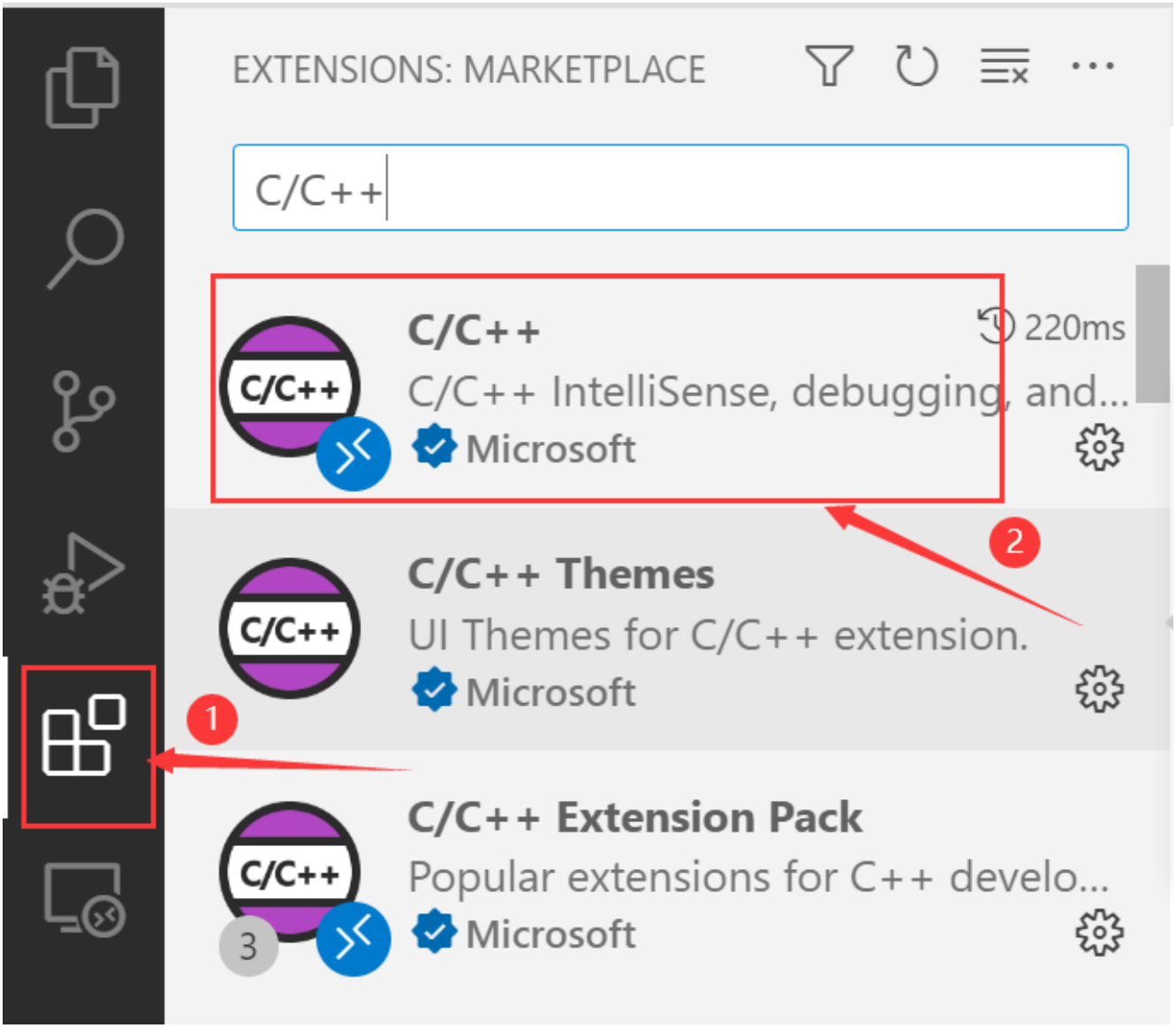
1. 打开VSCode，点击左下角的 "Open a Remote Window" 按钮。
2. 在弹出的下拉菜单中选择 "Connect to Host..." 。
3. 在输入框中输入远程计算机（Ubuntu 服务器）的 `用户名@IP地址` （如：`usr@192.168.221.131`）。
4. 一路回车直到提示输入密码。
5. 等待连接成功后，就能连接远程计算机的文件系统。





五、配置 C++ 开发环境

1. 打开VSCode。
2. 按下快捷键 `<Ctrl-Shift-x>` 或点击侧边栏的 "Extensions" 搜索插件。
3. 搜索并安装 C/C++ 相关插件。



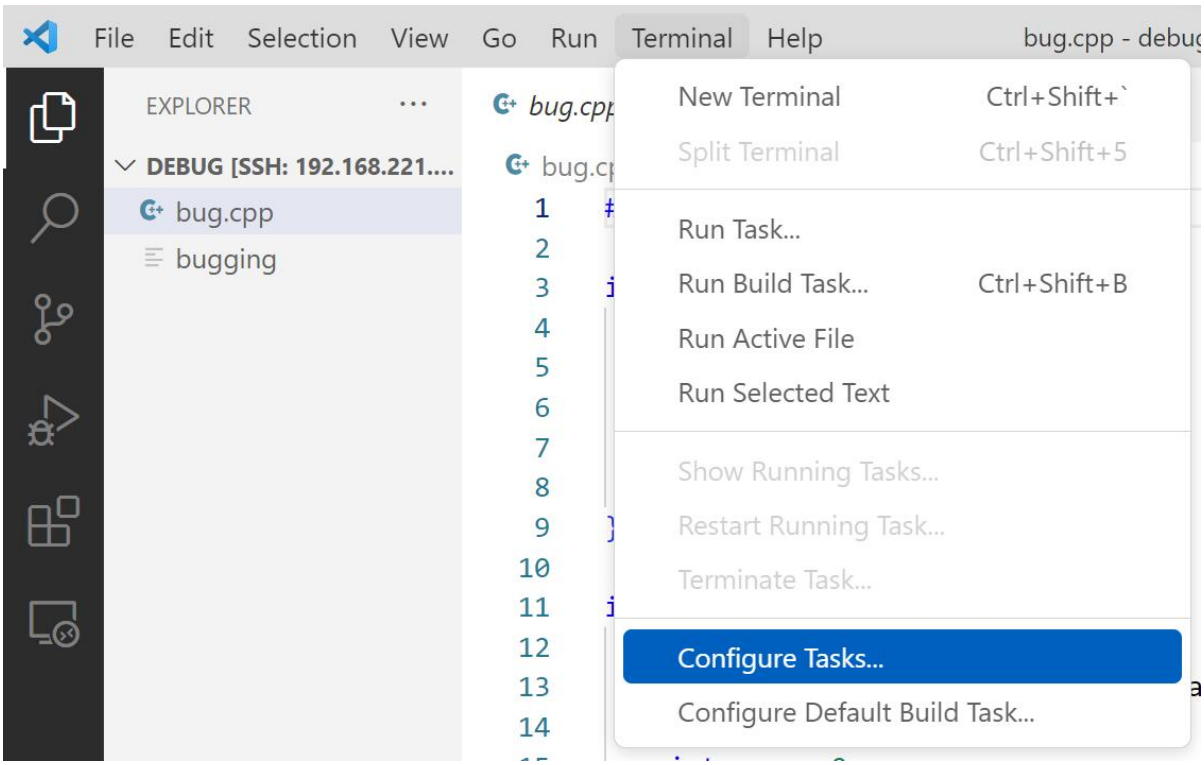
至此，我们已经基本完成了 VSCode C++ 环境的配置。

在 VSCode C++ 环境中，每个项目是以文件夹的形式存在的。可以像使用其它 IDE 环境那样对项目中的源文件进行添加、修改、保存、删除操作。然后通过命令行的方式编译程序，如：`g++ filename.cpp -o output`。

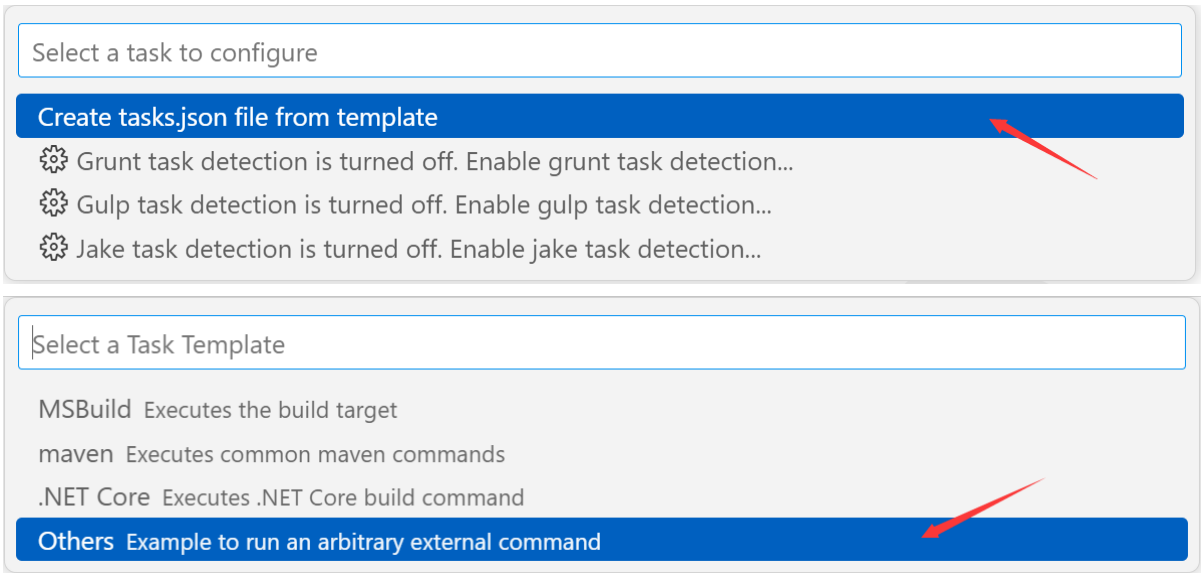
六、进一步配置 C++ 开发环境

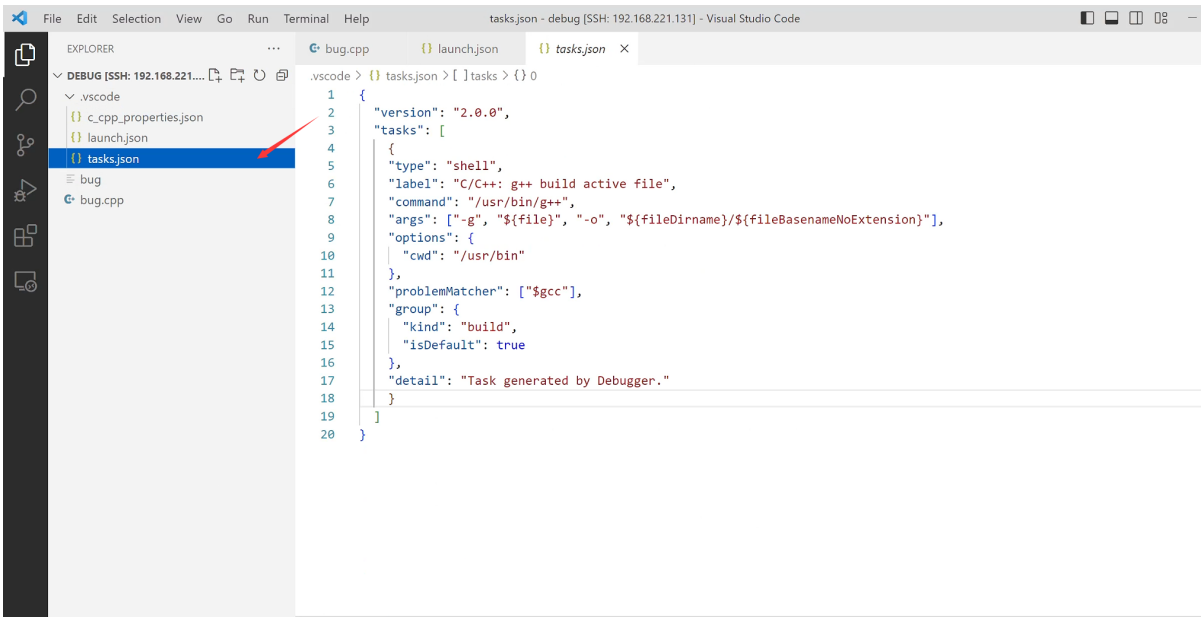
接下来，我们可以通过添加和修改 `.vscode` 目录下的几个配置文件来让 VSCode 上的 C++ 开发更加方便。

- 1. `tasks.json` 文件配置: `tasks.json` 用于告诉 VSCode 应该怎么编译源文件。
  - 从模版生成 `tasks.json` 文件（也可以直接手动添加）。
    - 在菜单栏点击配置任务，如图所示：



- 在弹出框中依次选择 "Create tasks.json file from template" - "others"。此时项目目录的 `.vscode` 目录下会出现 `tasks.json` 文件：





- 在文件中配置具体的 task, 官方给出的 task 样例如下：

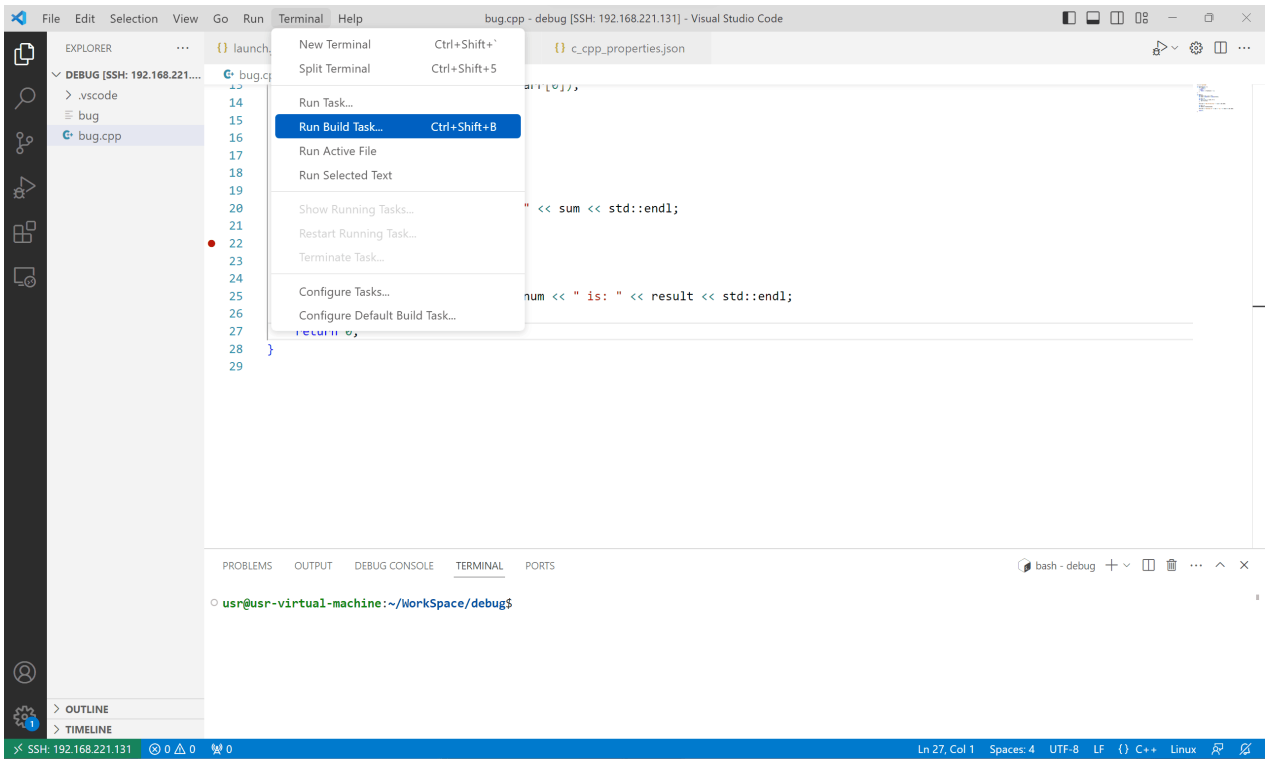
```
{
  "version": "2.0.0",
  "tasks": [
    {
      "type": "shell",
      "label": "C/C++: g++ build active file",
      "command": "/usr/bin/g++",  // 编译源文件使用的命令
      "args": ["-g", "${file}", "-o",
"${fileDirname}/${fileBasenameNoExtension}"], // 命令行参数
      "options": {
        "cwd": "${workspaceFolder}"
      },
      "problemMatcher": ["$gcc"],
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "detail": "Task generated by Debugger."
    }
  ]
}
```

- 对该文件的解释如下：
  - version: 配置文件的版本，目前为 "2.0.0"。
  - tasks: 一个数组，包含了所有的任务配置信息。
  - type: 任务类型，这里是 "shell"，表示使用 shell 命令进行编译。
  - label: 任务标签，用于在 VS Code 中显示任务名称。
  - command: 要执行的命令，这里是 "/usr/bin/g++"，表示使用 g++ 编译器进行编译。

        - args: 命令行参数，这里包含了四个参数："-g" 表示生成调试信息，"${file}" 表示当前打开的文件名，"-o" 表示输出文件名，"${fileDirname}/${fileBasenameNoExtension}" 表示输出文件的路径和名称。
        - options: 选项，包含一个 cwd 字段，表示在哪个目录下运行编译命令，这里设置为项目目录。
        - problemMatcher: 问题匹配器，这里使用了 "$gcc"，表示使用 GCC 编译器的输出来匹配错误信息。
        - group: 任务组，包含一个 kind 字段，表示任务的种类，这里是 "build"，表示构建任务，和一个 isDefault 字段，表示该任务是否是默认任务，这里设置为 true，表示该任务将作为默认构建任务。
        - detail: 任务的详细描述信息。
    - 这里再给出一个常见的 task.json 配置，一般用于使用 make 编译多个文件的情况：

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "type": "cppbuild",
          "label": "构建",
          "command": "make",
          "args": [],
          "options": {
              "cwd": "${workspaceFolder}"
          },
          "problemMatcher": [
              "$gcc"
          ],
          "group": {
              "kind": "build",
              "isDefault": true
          }
      }
  ]
}
```

- 配置好 task.json 之后，就可以在 VSCode 的菜单栏一键进行项目构建工作（也可以使用快捷键 `<Ctrl-Shift-b>`），如图：



- 2. `c_cpp_properties.json` 文件配置: 这个文件用于配置 C/C++ 相关信息，可以告诉 VSCode 使用的编译器、查找 include 文件的路径以及 C 文件和 CPP 文件的解析标准等，帮助 VSCode 的自动代码分析工具 IntelliSense 进行语法检查。
  - 在 VSCode 中，在某个打开的 C++ 文件下，按下 `Ctrl+Shift+P` 键，选择 `C/C++: Edit Configurations (JSON)` 即可在 `.vscode` 下创建 `c_cpp_properties.json`。
  - 在文件中配置相关的选项，默认配置如下：

```
{
  "configurations": [
    {
      "name": "Linux",
      "includePath": ["${workspaceFolder}/**"], // include 文件路径，表示在项目目录下查找
      "defines": [],
      "compilerPath": "/usr/bin/gcc", // 编译器路径
      "cStandard": "c11", // C 文件标准
      "cppStandard": "c++17", // CPP 文件标准
      "intelliSenseMode": "linux-gcc-x64"
    }
  ],
  "version": 4
}
```

至此，我们已经成功通过远程连接的方式在远程主机上搭建了C++开发环境。

1.2 GDB的使用

1.2.1 实验目的

了解并使用 gdb 调试器来调试 C++ 程序，以帮助定位和解决程序中的错误。

1.2.2 实验内容与要求

要求

- 熟悉 gdb 常用命令。
- 利用 gdb 调试器对程序进行调试。
- 在 VSCode 图形界面中使用 gdb 进行调试。

内容

- 注意：gdb程序调试的对象是可执行文件。
- 注意：为使用gdb，在编译时需要加上 `-g` 参数以包含调试信息。

一、常用命令

打断点

- break

  `b [文件:] 行号 [if 条件]`

  `b [类名: | 文件:] 函数名`

  `b [类名: | 文件:] 函数名带参数`

- watch

  `watch expression`

查看断点

- info

  `i b [断点编号]`

删除断点

- delete (以断点序号为关键字删除断点)

  `delete [断点编号 | 断点范围]`

- clear (以断点名称为关键字删除断点)

  `clear 函数名`

  `clear 函数名带参数`

  `clear [文件:]行号`

使能断点

- enable

  `ena [断点编号 | 断点范围]`
- disable

  `dis [断点编号 | 断点范围]`

启动

- run

  `r`

暂停

- `<Ctrl-c>`

继续执行

- continue

  `c`

停止

- kill

  `kill`

显示程序当前位置

- where

  `where`

退出gdb

- quit

  `q`

单步进入

- step (语言级)

  `s`
- stepi (汇编级)

  `si`

单步

- next (语言级)

  `n`
- nexti (汇编级)

  `ni`

跳出当前函数

- finish

  `finish`

查看源码

- list

  `l`：显示当前断点处的代码，往后显示。

  `l -` ：往前显示代码。

  `l [[文件名:] | 行号]`：显示指定行号代码。

  `l [函数名]`：显示执行函数的代码。

查看变量

- print

  `p [文件名:] 变量名`

- display (每次停止运行时均显示 expressino 的值)

  `display expression`

- x (显示内存中的值)
  `x /nfu`：以格式化方的方式查看指定地址的内存
  - n：单位数量
  - f：显示格式

    > x 按十六进制格式显示变量
    > u 按十进制格式显示无符号整型
    > o 按八进制格式显示变量
    > t 按二进制格式显示变量
    > a 按十六进制格式显示变量
    > i 指令地址格式
    > c 按字符格式显示变量
    > f 按浮点数格式显示变量

  - u：单位大小

    > b 单字节
    > h 双字节
    > w 四字节
    > g 八字节

查看变量类型

- ptype

  `ptype 变量名`

查看函数堆栈

- backtrace

  `bt [[-]栈帧数量]`

查看栈帧

- frame

  `f [帧编号]`

在栈帧间移动

- up

  `up [栈帧数量]`

- down

  `down [栈帧数量]`

查看当前栈帧局部变量

- info

  `i local`

查看线程信息

- info

  `i thread`

切换调试线程

- thread

  `thread 线程编号`

二、在命令行中使用gdb常用命令调试一个正常程序

源码如下：

```
#include <iostream>

int factorial(int n) {
    if (n == 0) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);
```

```
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }

    std::cout << "Sum of the array: " << sum << std::endl;

    int num = 5;
    int result = factorial(num);

    std::cout << "Factorial of " << num << " is: " << result << std::endl;

    return 0;
}
```

调式步骤：

1. 编译程序时添加调试信息：`g++ -g -o debug_example debug_example.cpp`。
2. 启动gdb调试：`gdb debug_example`。
3. 设置断点：在 `main` 函数的开头，以及在 `factorial` 函数的返回语句前设置断点。

   `break main`

   `break factorial if n == 1`
4. 查看断点：`i b`。

   ```
   (gdb) i b
   Num     Type           Disp Enb Address            What
   1       breakpoint     keep y   0x0000555555555224 in main() at bug.cpp:11
           breakpoint already hit 1 time
   4       breakpoint     keep y   0x00005555555551f8 in factorial(int) at bug.cpp:4
           stop only if n == 1
   ```
5. 运行程序：输入 `run`，程序会在停在断点处。
6. 查看变量：查看 `arr` 数组、`size`、`sum`、`num`等变量的值。此时变量的值未被初始化。

   `print arr`

   `print size`

   `print sum`

   `print num`

```
Breakpoint 1, main () at bug.cpp:11
11      int main() {
(gdb) print arr
$1 = {-136154936, 32767, -136155512, 32767, -136156864}
(gdb) p size
$2 = 32767
(gdb) p sum
$3 = 32767
(gdb) p num
$4 = -137239030
```

7. 单步调试：输入 `next` 命令单步执行 `main` 函数，并使用 `print` 命令查看变量值。

```
(gdb) n
12          int arr[] = {1, 2, 3, 4, 5};
(gdb) n
13          int size = sizeof(arr) / sizeof(arr[0]);
(gdb) n
15          int sum = 0;
(gdb) n
16          for (int i = 0; i < size; i++) {
(gdb) p arr
$1 = {1, 2, 3, 4, 5}
(gdb) p size
$2 = 5
(gdb) p sum
$3 = 0
```

8. 输入 `watch` 命令来监视 `sum` 变量的值，当 `sum` 变化时，触发断点。

```
(gdb) watch sum
Hardware watchpoint 3: sum
(gdb) c
Continuing.

Hardware watchpoint 3: sum

Old value = 0
New value = 1
main () at bug.cpp:16
16          for (int i = 0; i < size; i++) {
```

9. 输入 `continue` 命令继续函数执行，由于前面设置的条件断点，会进入 `factorial` 函数并触发断点。

```
(gdb) c
Continuing.
Sum of the array: 15

Breakpoint 2, factorial (n=1) at bug.cpp:4
4           if (n == 0) {
```

10. 输入 `backtrace` 命令查看当前的堆栈信息。

```
(gdb) backtrace
#0  factorial (n=1) at bug.cpp:4
#1  0x0000555555555212 in factorial (n=2) at bug.cpp:7
#2  0x0000555555555212 in factorial (n=3) at bug.cpp:7
#3  0x0000555555555212 in factorial (n=4) at bug.cpp:7
#4  0x0000555555555212 in factorial (n=5) at bug.cpp:7
#5  0x00005555555552d1 in main () at bug.cpp:23
```

11. 输入 `continue` 命令继续执行程序，直到遇到下一个断点或程序结束。
12. 输入 `quit` 命令退出GDB调试器。

三、在命令行中使用 gdb 命令行调式一个有错的 C++ 程序

源码如下：

```
#include <iostream>
#include <vector>
```

```
class BaseClass {
public:
    BaseClass(int value) : data(value) {}

    virtual void printInfo() {
        std::cout << "BaseClass: " << data << std::endl;
    }

    void divideBy(int divisor) {
        data /= divisor;
    }

protected:
    int data;
};

class DerivedClass : public BaseClass {
public:
    DerivedClass(int value) : BaseClass(value) {}

    void printInfo() override {
        std::cout << "DerivedClass: " << data << std::endl;
    }
};

int main() {
    int size;
    std::cout << "Enter the number of elements: ";
    std::cin >> size;

    std::vector<BaseClass*> objects;

    for (int i = 0; i < size; ++i) {
        objects.push_back(new DerivedClass(i + 1));
    }

    for (int i = 0; i <= size; ++i) { // 此处应该是 < 而不是 <=
        objects[i]->printInfo();
        objects[i]->divideBy(i); // 此处应该是 i + 1 而不是 i
    }

    for (BaseClass* obj : objects) {
        delete obj;
    }

    return 0;
}
```

调试步骤：

1. 对源码进行编译，生成可执行文件 bug。
2. 运行 bug，发现错误。

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

usr@usr-virtual-machine:~/WorkSpace/debug$ ./bug
Enter the number of elements: 3
DerivedClass: 1
Floating point exception (core dumped)
usr@usr-virtual-machine:~/WorkSpace/debug$
```

3. 运行 `gdb bug` 命令，装入可执行文件。
4. 输入 `break main` 在 `main` 函数的开头设置一个断点。

```
(gdb) break main
Breakpoint 1 at 0x12f6: file /home/usr/WorkSpace/debug/bug.cpp, line 29.
(gdb) i b
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x00000000000012f6 in main() at /home/usr/WorkSpace/debug/bug.cpp:29
```

5. 输入 `run` 命令运行程序。
6. 输入 `list` 命令来显示当前停止的位置周围的代码，确认代码位置。

```
(gdb) list
24          void printInfo() override {
25              std::cout << "DerivedClass: " << data << std::endl;
26          }
27      };
28
29      int main() {
30          int size;
31          std::cout << "Enter the number of elements: ";
32          std::cin >> size;
33
```

7. 输入 `next` 命令直到进入第二个循环体。

```
40          for (int i = 0; i < size; ++i) { // 此处应该是 < 而不是 <=
(gdb) n
41              objects[i]->printInfo();
(gdb) list
36          for (int i = 0; i < size; ++i) {
37              objects.push_back(new DerivedClass(i + 1));
38          }
39
40          for (int i = 0; i < size; ++i) { // 此处应该是 < 而不是 <=
41              objects[i]->printInfo();
42              objects[i]->divideBy(i + 1); // 此处应该是 i + 1 而不是 i
43          }
44
45          for (BaseClass* obj : objects) {
```

8. 输入 `print i`，`print size`，`print *object[i]` 命令，分别查看变量的值和当前对象的信息。

```
(gdb) p i
$6 = 0
(gdb) p size
$7 = 3
(gdb) p *objects[i]
$8 = {_vptr.BaseClass = 0x555555558ce8 <vtable for DerivedClass+16>, data = 1}
(gdb)
```

9. 输入 `continue` 命令继续运行，直到第一处错误。
10. 从命令行输出看到是执行除法计算时出错，判断出错原因最有可能是除数为 0。
11. 输入 `print divisor` 打印 `divisor` 变量进行验证，不出所料为 0。

```
(gdb) c
Continuing.

Program received signal SIGFPE, Arithmetic exception.
0x00005555555555c1 in BaseClass::divideBy (this=0x55555556c6d0, divisor=0) at /home/usr/WorkSpace/debug/bug.cpp:
13
13              data /= divisor;
(gdb) p divisor
$9 = 0
```

12. 输入 `up` 命令，返回上层函数，发现除数是变量 `i` 传进来的，要修改该错误需将 `i` 改为 `i + 1`。

```
(gdb) up
#1  0x00005555555553da in main () at /home/usr/WorkSpace/debug/bug.cpp:42
42              objects[i]->divideBy(i); // 此处应该是 i + 1 而不是 i
(gdb) list
37              objects.push_back(new DerivedClass(i + 1));
38          }
39
40          for (int i = 0; i < size; ++i) { // 此处应该是 < 而不是 <=
41              objects[i]->printInfo();
42              objects[i]->divideBy(i); // 此处应该是 i + 1 而不是 i
43          }
44
45          for (BaseClass* obj : objects) {
46              delete obj;
```

13. 修改该错误后重新编译，再次进入调试，直接输入 `run`，程序在第二处错误处停止。

```
(gdb) c
Continuing.
Enter the number of elements: 3
DerivedClass: 1
DerivedClass: 2
DerivedClass: 3

Program received signal SIGSEGV, Segmentation fault.
0x00005555555553aa in main () at /home/usr/WorkSpace/debug/bug.cpp:41
41              objects[i]->printInfo();
```

14. 输入 `list` 命令定位出错处代码，发现是在对第三个 `objects` 对象调用 `printInfo` 方法时出现错误。
15. 输入 `print objects[i]` 查看当前对象的信息。

```
(gdb) p objects[i]
$5 = (BaseClass *) 0x0
(gdb) p *objects[i]
Cannot access memory at address 0x0
```

16. 发现 `objects[i]` 对象为空，输入 `print i` 和 `print objects.size()` 命令进一步比较数组长度和当前下标。

```
(gdb) p i
$6 = 3
(gdb) p objects.size()
$7 = 3
```

17. 我们发现 `objects[i]` 对象为空是因为数组访问越界。通过对比代码我们发现我们只初始化了 `0 ~ size-1` 个 `DerivedClass` 类型的对象，但是在第二处循环我们却尝试获取第 `size` 个对象，并调用其方法，这显然会出错。



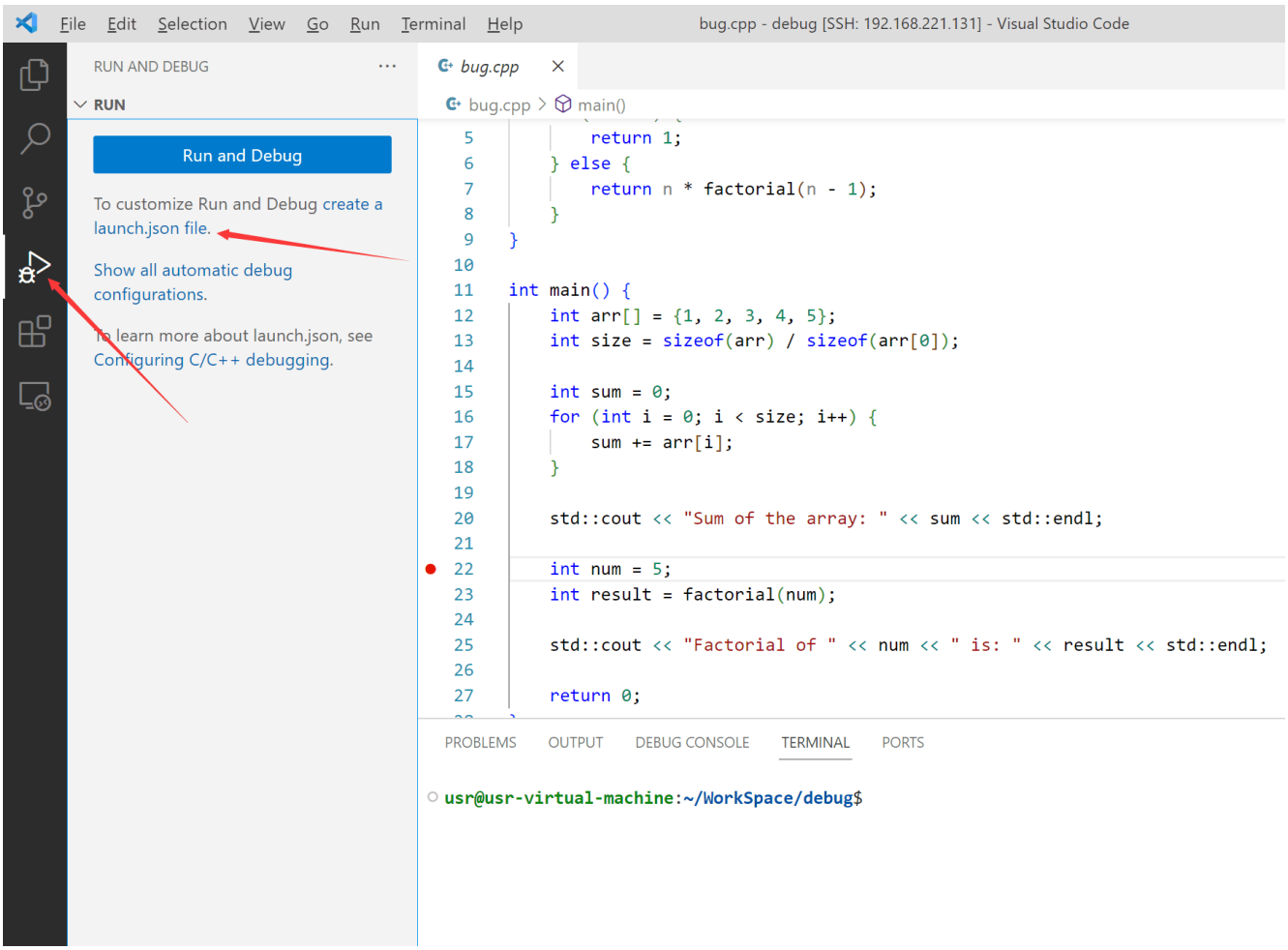
18. 将 40 行处的 `<=` 改成 `<` 后再次编译调试，发现程序正常结束，至此完成调试。

```
(gdb) run
Starting program: /home/usr/WorkSpace/debug/bug
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter the number of elements: 3
DerivedClass: 1
DerivedClass: 2
DerivedClass: 3
[Inferior 1 (process 12788) exited normally]
```

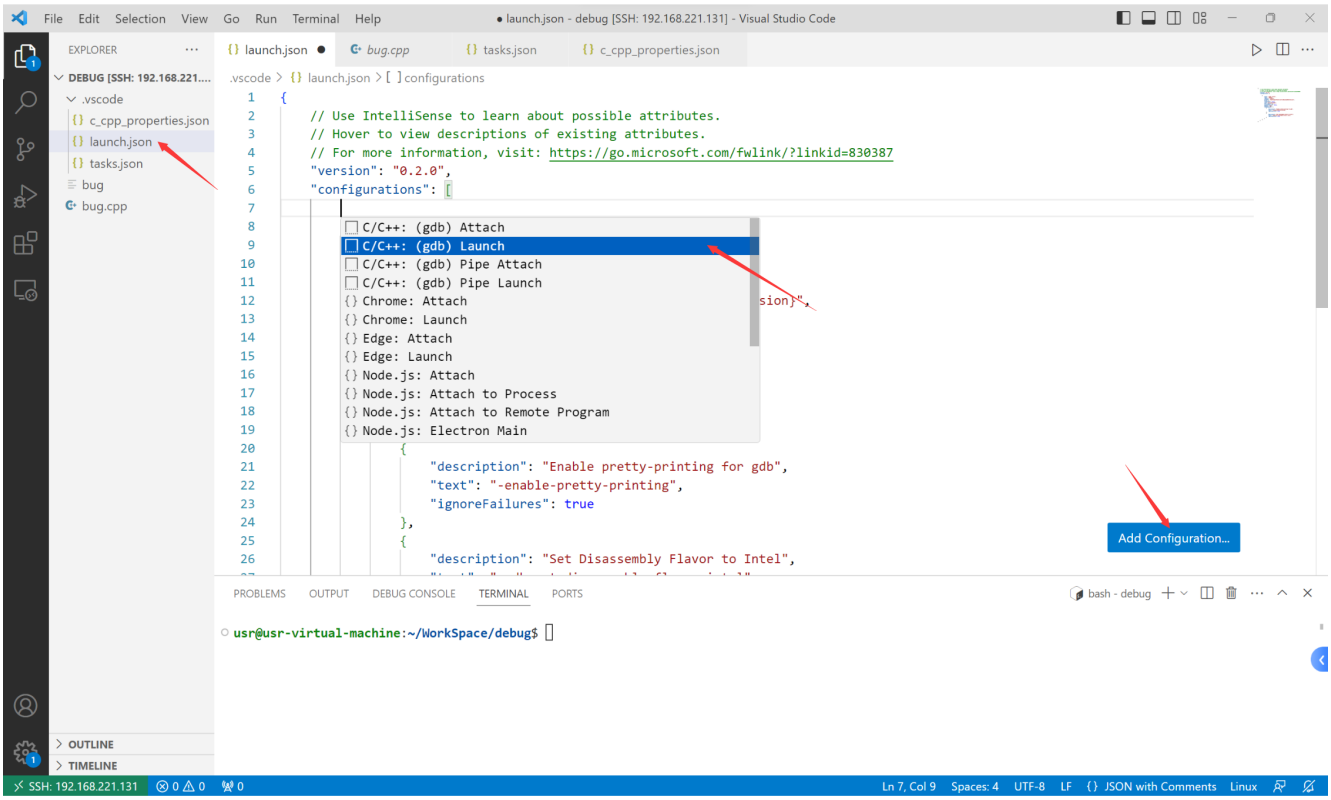
四、VSCode 图形化调试C++程序

首先我们需要配置 `.vscode` 目录下的 `launch.json` 文件，该文件用于配置 VSCode 的调试器。

1. 根据下图所示，生成 `launch.json` 模版（也可以直接创建该文件）。



2. 转到 `launch.json` 文件，在其右下角会有 `Add Configurations` 按钮。选择 `C/C++: (gdb) Attach` 或 `C/C++: (gdb) Attach` (launch模式由 VSCode 来启动一个独立的具有 debug 模式的程序；attach 模式附加于一个已经启动的程序)。

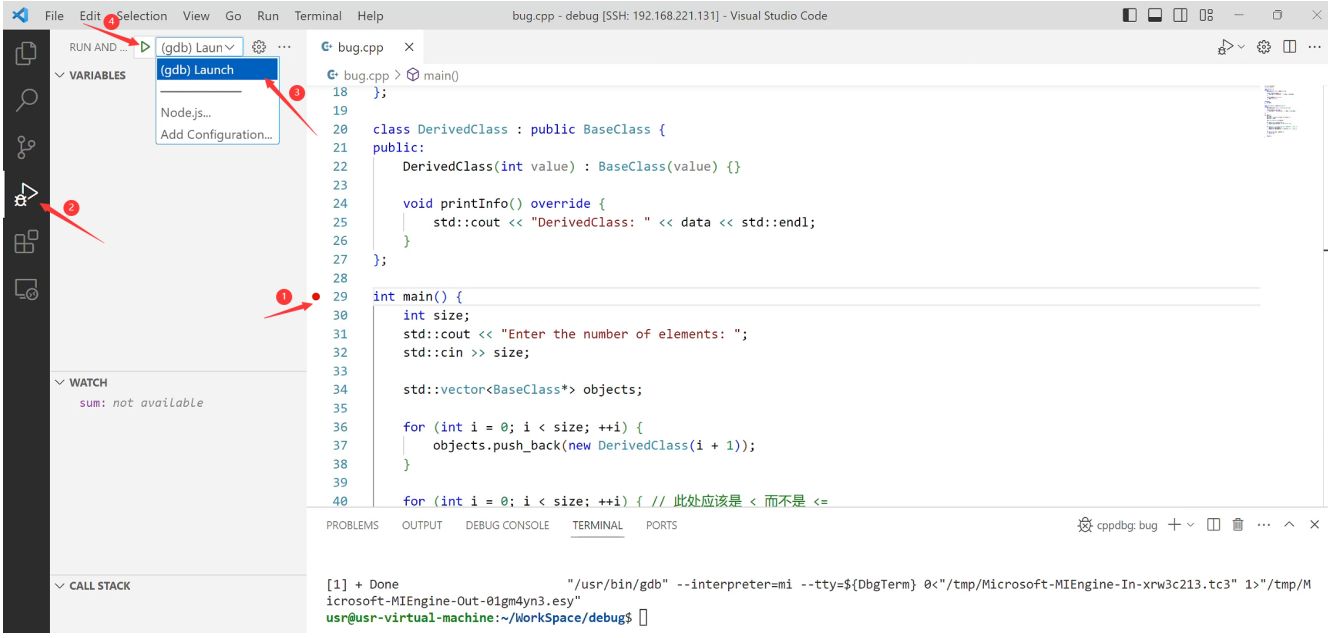


3. 配置文件如下所示：

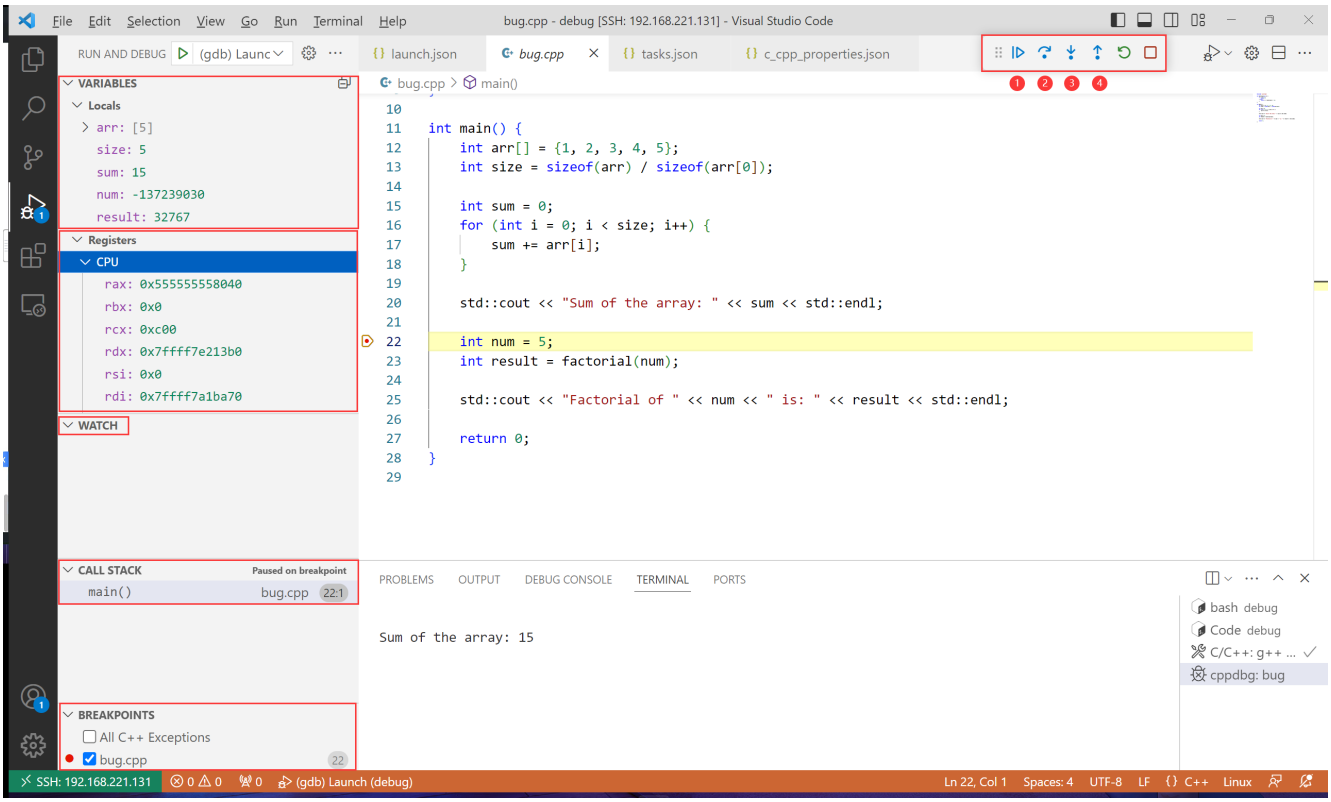
```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "(gdb) Launch",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/${fileBasenameNoExtension}", // 需要运行的是当前项目目录中，目前打开文件的除去拓展名的文件名
      "args": [],  // 设置启动参数
      "stopAtEntry": false, // 设为 True 则会在打开控制台后停滞，暂不执行程序
      "cwd": "${fileDirname}",  // 当前工作路径
      "environment": [],  // 设置环境变量
      "externalConsole": false,
      "MIMode": "gdb",  // 表示 VSCode 将连接的调试器。必须设置为 gdb 或 lldb. 这里表示使用GDB调试
      "setupCommands": [
        {
          "description": "Enable pretty-printing for gdb",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ]
    }
  ]
}
```

更多配置选项见https://code.visualstudio.com/docs/cpp/launch-json-reference。

4. 首先在待调试程序上打上一个断点，然后点击左侧边栏的 Debug 按钮，选择 "(gdb) Launch" (此处应该与配置文件中的 `name` 字段对应)，点击左侧 `Start Debugging` (或使用快捷键 `F5`)进入调试。



5. 调试界面的左侧窗口，由上到下分别可以查看变量值、寄存器值、watch 的表达式、函数调用栈以及断点信息。



6. 上图中另外 4 个按钮从前三个分别对应 gdb 命令行调试中的 `continue`，`next`，`step`，最后一个 `step out` 表示从当前函数跳出。